

CS4203 Practical 2 - The Shared Files

210016688

Introduction

For this practical, I was asked to design and implement a client-server model for a cryptographically secure file server that allows users to securely store, retrieve, and share files with a zero-trust approach. This means that, at a minimum, the provider does not have access to the unencrypted data or the encryption keys. My final implementation provides a client and server program with a simple terminal interface. The programs also use basic certificate authority implementation for the use of server-side and client-side certificates.

Threat Model

This section presents a basic threat model for the cryptographically secure file server. It focuses on primary security concerns related to data confidentiality and integrity, with the design section expanding in areas. The design addresses the core threats identified here while balancing usability and security.

When designing the threat model for our program, we identify two primary assumptions that allow us to establish a reasonable security baseline. First, we assume a trusted client environment. This assumption allows us to focus on server-side and network-related threats rather than individual user vulnerabilities, which would be too large and abstract to map. Furthermore, these forms of attacks would require direct access to a user and their device. The second assumption is the existence of a secure and mutually trusted certificate authority. For the sake of this practical, I have implemented a basic CA that we use to generate and verify client-side authentication certificates and server domain certificates used in TLS. While this CA enables secure client-server communication, it is simplified for this project and does not support advanced functions such as certificate revocation or extensive identity verification.

Our primary concern in this project is preventing unauthorised access to data, meaning that unauthorised users, including the cloud provider, could gain access to the stored data. By using end-to-end encryption, we ensure that only users with the correct decryption keys can access the data. By performing the encryption and decryption on the client side, not giving keys to the server, as required in zero-knowledge, whether data is attacked in transit, in memory, or at rest, the data can be assumed to be secure.

A critical risk when dealing with remote data storage is that data could be intercepted during transmission between the client and server. Taking advantage of Transport Layer Security (TLS) to encrypt the data stream, preventing man-in-the-middle attacks. The use of TLS also makes it harder to perform attacks such as replay attacks due to built-in timestamps and ordering. It also provides the ability to encrypt any metadata that would need to be sent to a server along with the encrypted data. There are further risks in how that data is stored at rest as well as being processed. Whilst data travelling between the server and client might be secure, raw data stored on the server could be available to the service provider as well as any malicious attackers. We have similar concerns about monitoring the server whilst data is being processed.

Key compromise and user impersonation represent additional threats to the system. If an attacker obtains access to encryption keys, they could decrypt stored files or impersonate legitimate users. To counter this, the system implements strict key management practices: private keys are stored solely on the client, and symmetric keys are encrypted with the recipient's public key so that only authorised users can decrypt them. To facilitate key distribution, public keys are linked to user-friendly identifiers, such as email addresses. Each user's identity is confirmed through a client authentication certificate signed by the CA, which enables both the server and clients to validate a user's public key and username pairing. By associating public keys with verified email addresses, we can introduce multi-factor authentication (MFA) options, such as one-time passwords or magic links, which add another layer of security against social engineering and phishing attacks.

A compromised server is another potential risk, as it could expose metadata about stored files or facilitate unauthorised access to encrypted data. To address server compromise and impersonation, the server is designed as a zero-knowledge entity that stores only encrypted data, with no access to decryption keys. This design choice limits the information that could be exposed even during a server breach. Additionally, the system minimises metadata storage to reduce potential leakage. Server impersonation is

mitigated by using certificates signed by the CA, enabling clients to verify the server's authenticity before sharing sensitive data.

Certain threats fall outside the scope of this project but are worth noting. Denial of Service (DoS) attacks, for instance, can disrupt availability but do not expose data directly. Additionally, while the measures implemented here prevent data exposure, they do not address data loss; an attacker with server access or even the cloud provider could delete files. I determined that implementing data loss prevention or backup mechanisms was outside this practical's scope, but given more time, it would be an interesting extension to the practical.

Design Decisions

My program makes use of Python to implement a client-server model and a certificate authority, leveraging Python's robust and well-documented cryptography library, which supports X.509 certificate management. Client-server communication is secured via a TLS connection using a text-based protocol similar to HTTP. I chose not to use HTTPS, as the project does not require the extensive feature set of HTTPS and, in some places, requires longer-living connections to transfer data back and forth, which is not a function of HTTP. Additionally, designing a custom header structure allows for efficient data transfer while enabling built-in signatures for message validation. To support client authentication certificates and server certificates for TLS, I implemented a rudimentary certificate authority that takes a common name and a public key and creates a signed certificate.

Identity and Key Distribution

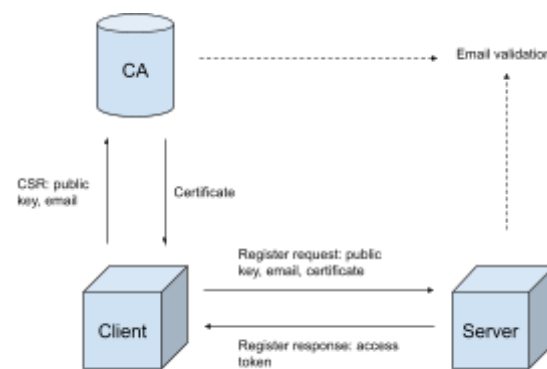
In this model, each user's identity is defined by their email address and an RSA public key. An RSA key pair allows for both the signing and encryption of data. It is larger when compared to other keys, such as ED25519, which is also well-supported in Python, but it does not provide encryption support, only signing. When first using the application, a user generates a public-private key pair, with the private key securely stored locally on their device and never uploaded or shared. The private key is encrypted using the email address and a local password for additional security. While not strictly necessary due to the assumed secure client environment, this additional layer serves as multi-factor authentication, requiring both device access and a password. The email address and public key are used to perform a certificate signing request (CSR) with our Certificate Authority (CA), producing a certificate that links the user's public key—essential for data-sharing—with their email address. The user then registers on the server through a *register* request, submitting their email (serving as a username), public key, and certificate in the header. This setup enables easy and secure key distribution, as the email address is simpler and more reliable for communication than a long public key, which could be easily miscommunicated or misentered.

The server acts as a keystore, providing users with others' public keys when needed. Alongside each public key, the server also distributes the corresponding certificate, signed by a mutually trusted CA, to verify the authenticity of the key and prevent attacks like keystore poisoning, where malicious users might attempt to substitute fake keys. This design relies on the CA to verify each email address, ideally through methods like one-time passwords or magic links, to ensure the legitimacy of each identity before signing the certificate. To help resolve any issues such as poisoning or accidental data loss, the certificate is sent on subsequent *login* requests so that an invalid certificate can be replaced. We do assume that the certificate authority performs an appropriate check for the email, such as a one-time password or a magic link when the certificate is signed, so it can claim that the identity is real.

While not a strict security vulnerability, this method does impact user privacy by tying each user's email address to their public key, effectively making the email a piece of publicly accessible information. As a result, our server maintains a list of valid email addresses, which could potentially be exploited. However, using email addresses as identifiers offers significant advantages: they provide a universally recognized and commonly used username format, making it easier for users to share their identity. Furthermore, email addresses allow for straightforward multi-factor authentication, enhancing security during sensitive operations.

Although I considered hashing the email addresses before storing them on the server, this approach would have limited benefits. Since the email address must be in plaintext when communicating with the Certificate Authority for verification, it would also appear as plaintext in the certificates. Thus, attempting to

obfuscate email addresses in server storage would only add unnecessary complexity without significantly enhancing privacy.



Network Level Security and Application Level Authentication

The program uses Transport Layer Security (TLS), a structured header design, and an access token system to secure data transfer and authenticate connections.

On server startup, the server first checks for an existing certificate and, if none is found, automatically requests one from the Certificate Authority (CA). This certificate is then used to establish secure TLS connections, encrypt data in-flight and prevent the interception of data. Whilst file data will already be encrypted, to maintain the zero-knowledge approach, by using TLS, the system not only secures the file data but also conceals metadata, such as the identity of the file sender, reducing exposure to potential attackers. TLS additionally provides built-in protection against additional types of attacks such as replay attacks through timestamping and ordering mechanisms. Moreover, the server's certificate acts as a form of network-level authentication, ensuring that clients connect to the correct server, thereby reducing the risk of man-in-the-middle attacks.

The system employs a custom header structure for server requests to strengthen application-level authentication. This header format consists of text-based key-value pairs preceded by a 4-byte field specifying header length, a 4-byte field for the signature length, and the signature itself. The signature is generated by hashing the header content (using SHA-256) and signing it with the user's private key, ensuring that only authorised users can send valid requests. In addition to signed headers, the system uses access tokens for secure user authentication. When users log in, they generate an access token, stored in memory on their device and signed by the server's private key. The access token includes the user's public key and is sent in base64 form in the header of subsequent requests, serving as a form of session authentication. This is intended to be used in combination with a method of email authentication, such as one-time passwords or magic links, providing a form of multi-factor authentication.

These measures address the primary concerns identified in the Threat Model, specifically by reducing the risk of data interception and man-in-the-middle attacks and ensuring that network and application-level security are maintained at a high standard.

The combination of hashed and signed headers with access tokens requires direct client access to a user's private key to make requests and an email confirmation to interact with the server. Hashing the header also ensures header integrity, further protecting against unauthorised modifications. Similarly, any data included in the body of the message is also hashed for integrity verification. These measures, along with the use of client authentication certificates described above, help to address any concerns over user impersonation as discussed in the Threat Model.

Client-Server Model

The client is responsible for encrypting data before uploading it to the server, which is required to maintain the zero-knowledge approach. The solution involves encrypting each file with a symmetric key using AES-256, which is efficient for handling large files. For each authorised user, this symmetric key is then encrypted with their public key, allowing both the encrypted file and the encrypted symmetric keys to be stored on the server. This approach simplifies access distribution, as sharing a file with a new user only requires encrypting the symmetric key with their public key rather than re-encrypting the entire file. Additionally, revoking access is

straightforward, as we remove that user's encrypted symmetric key from the server, preventing further access to the file. The separation of client-side encryption from server storage is crucial in maintaining data confidentiality against potential server breaches or unauthorised access. This is the basics of my client-server model.

Access Control Lists

The program implements an access control mechanism to securely manage file access, allowing users to share and manage file keys. This ensures users can only access files they have explicit permission to view or modify. Various approaches were considered before finding a solution that balances security, usability, and adherence to a zero-knowledge model.

An initial approach involved giving users decryption keys and lists of accessible files while allowing open downloads. This reduced server-side metadata but posed risks, as unrestricted access could, in rare cases, compromise sensitive data despite encryption. Another design option was a token-based authentication system, where JWT-like tokens represent access rights. This minimised stored metadata but introduced complexity in managing and revoking tokens, adding server processing overhead.

The final design combines elements of both an access control list (ACL) and a token-based approach. This hybrid system allows the server to confirm user permissions while minimising and obscuring sensitive metadata stored on the server. The granularity of ACLs allowed my final implementation to have three levels of permissions: owner, editor and viewer. The owner with all permissions, the editor the ability to change the end file and share permissions, and the viewer only download the file.

When a user uploads a file, it is encrypted client-side and stored as a randomly named blob on the server. The server then creates an ACL for the file, which maps each authorised user to their encrypted symmetric key and access level. Each ACL is stored as an encrypted blob in the server's file directory with a randomly generated name. The server provides the client with the ACL and blob filenames, which the client must present in future requests to prove access. This system does present potential risks regarding metadata exposure. While the server cannot access the file contents, it can see the ACL structure, which reveals user relationships and permissions. And potentially by editing this data we could introduce social engineering attacks over claimed access but lost credentials. Nevertheless, the malicious attackers are prevented from impersonating other users or tampering with data, as all user identities are established through a certificate authority and access tokens. To enhance metadata privacy, in the final variation, the ACL itself is encrypted with a server-generated symmetric key, while usernames within the ACL are securely hashed to prevent plaintext visibility. Although this adds a layer of obscurity to protect access data at rest, it depends on the server managing keys securely. Any exposure of server memory or an attacker systematically compromising the generated keys could still reveal sensitive relationships within the ACL. But these forms of attack require an attack at processing, whilst the server has the data in memory, arguably a harder form of attack, at least from an outside party.

The program uses encrypted notifications allowing each client to locally track which files they can access. Rather than storing file listings on the server, each user's client application keeps an encrypted record of accessible files, the corresponding ACL, the encryption key for the ACL, and maps hashed usernames to original identities. Encrypted notifications are stored in binary format in a file corresponding to a user's hashed username. When a user goes to the homescreen in the client application, it automatically requests the notifications for its account from the server and parses them. When a file is shared with a new user, the system generates two types of notifications: one that contains the necessary access information for the new user and another notifying current users of the new user's hashed identity. If access is revoked, the program updates the ACL by removing the user from the file's permissions list and sending them a notification to indicate their access is no longer valid. Each time a user receives a notification, they verify it by attempting to access the relevant file before updating their local records.

This access control design prioritises privacy and secure sharing while maintaining the zero-knowledge principle, as outlined in the threat model, by ensuring sensitive data and relationships remain as obscured as possible from the server.

Usability and Complexity Management

Whilst not directly a security issue, the usability of a program contributes to its security through accidental improper and insecure use of a program or potential social engineering attacks. That is why I have

created a simple terminal interface for the program, allowing users to perform all the operations from a single application. For this reason, I also made several simple scripts that start instances of the client and server, taking command line arguments to set the addresses and ports used.

Evidence of Functionality

The final result can be run using the scripts provided as outlined in the README. Below show screenshots of using the application. The first shows a successful upload of a user's own file, the second shows a user looking at the permissions of a shared file that it can edit and download.

```
[1 - bash] [2 - bash] [3 - bash] [4 - zsh]
Logged In As: user1@example.com
-----
Filename - Filesize - Updated - Owner
1. README.md - 6261 bytes - 2024-11-26 - user1@example.com

- Upload <filepath>
- Download <file number>
- Delete <file number>
- Share <file number>
- Replace <file number>
- Refresh
- Logout
Enter a command: 
```

```
[1 - bash] [2 - bash] [3 - bash] [4 - bash]
Logged In As: user1@example.com
-----
Access list for tidy.sh:
  user2@example.com (Owner)
  user1@example.com (Editor)

- Add <email>
- Remove <email>
- Back
Enter a command: 
```

Evaluation

Overall, I do believe that I was successful in completing the practical, designing and implementing a zero-knowledge file server, as shown in the final product. Not only is it able to share and revoke access, with three different permission levels, given a certain level of permissions, you can change the end file.

Whilst I do believe my client-server model provides a good starting ground for an example of a zero-knowledge server, there are still improvements to be made. This is especially true when it comes to data at rest attacks involving data loss. The best solution I can see to prevent attacks involving data loss would be to move beyond the client-server model and introduce a decentralised database.